4DV651

Project in Model-Based Development

Project Proposal

25HT
Yan Song
Oct. 12th, 2025

Objective

Build a Model-Driven Engineering (MDE) solution that schedules airport ground support equipment (GSE) to service flights during turnarounds with minimal delays and efficient resource usage. The system will generate solvable scheduling instances from domain models, invoke an external optimizer, and provide schedule assignment, maintaining up-to-date rosters throughout the operating day.

Hypothetical airport setting: "Cedar Ridge Regional Airport (YCR/CYCR)" with 24 stands (A1–A24), one main equipment depot, and peak traffic of 12 movements/hour (Movement: one takeoff / landing performed by one aircraft). GSE types include belt loaders, baggage tugs, ground power units (GPU), catering trucks, fuel bowsers, passenger buses, and pushback tractors. Turnaround tasks follow standard precedence with regulatory constraints (e.g., no catering during fuelling).

Domain and Problem

Domain:

Airport apron operations and ground support logistics.

Problem:

Given a flight schedule and an airport layout, allocate and route a finite fleet of GSE to flight turnaround tasks subject to:

- Temporal constraints: task windows, durations, setup times.
- Sequence: deboarding - cleaning - catering - boarding; pushback at departure; fuelling rules.
- Resource constraints: unit exclusivity, compatibility (equipment type vs. task), driver shifts, maintenance windows.
- Spatial constraints: travel times on service road graph; depot/stand locations; temporary blockages.
- Safety/regulatory constraints: fuelling vs. catering mutual exclusion; stand occupancy limits.

Objectives:

- Minimize flight service lateness and regulatory violations.

- Minimize fleet size used and peak concurrent utilization.
- Minimize deadhead travel and waiting.
- Balance workload across units/shifts.

Models

Models and Metamodels

We define five metamodels:

AirportOpsMM (airport operations domain)

Concepts:

- Flight, Arrival/Departure, Turnaround, Stand.
- TaskType (Deboard, Clean, Cater, Fuel, BaggageIn/Out, GPU, Boarding, Pushback).
- Task with duration, earliest/latest time, mandatory/optional, and precedence edges.
- OperationalRules (e.g., fuelling vs. catering exclusion, minimum buffers).

EquipmentMM (equipment domain)

Concepts:

- EquipmentType (GPU, Tug, Bowser, etc.), Compatibility (TaskType and EquipmentType).
- EquipmentUnit with speed, capacity (1 task at a time), setup time, homing location, maintenance plan.
- OperatorShift with start/end, breaks, max continuous duty.

LayoutMM (apron layout domain)

Concepts:

- ApronGraph with Nodes (Stands, Depot, Maintenance bays) and Edges (directed; travel time; availability intervals).
- DistanceMatrix or on-demand travel-time service.

ScheduleProblemMM (solver input domain)

Concepts:

- Job (a turnaround task), JobWindow, JobDuration.

- ResourceType and ResourceUnit.
- Precedence, Incompatibility, Setup and Travel function travel(u, fromJob, toJob).
- ObjectiveWeights and ScenarioParameters (horizon, replan policy).

ScheduleResultMM (solver output domain)

Concepts:

- Assignment (Job ↔ EquipmentUnit), Start/End times, Lateness.
- Route (ordered Jobs per unit with travel legs).
- KPIs (OTP%, total deadhead, peak units used).

Domains, Metamodels, and Their Relations

Domains:

- Airport Operations (AirportOpsMM + LayoutMM).
- Equipment (EquipmentMM).
- Optimization Tool (ScheduleProblemMM, ScheduleResultMM).

Relations:

AirportOpsMM and EquipmentMM are independent but reference-compatible via TaskType <-> EquipmentType Compatibilities.

LayoutMM connects to AirportOpsMM via Stand and Node identities; it provides travel time semantics.

ScheduleProblemMM is derived (M2M) from AirportOpsMM + EquipmentMM + LayoutMM for a selected horizon.

ScheduleResultMM is produced independently by the tool and consumed by the platform via T2M.

AirportOps and Equipment models are updated (M2M) from ScheduleResultMM to annotate tasks and unit rosters with assigned times and routes.

Tool integration

We will integrate a Python solver to solve the schedules as a constraint programming problem. There are libraries like OR-Tools from Google, or python-constraint available, which are purpose made to solve such problems.

Integration strategy:

The platform generates a solver-ready artifact (JSON or similar, text.) from ScheduleProblemMM via M2T. Then the Python solver consumes this artifact and produces a text result conforming to a schema aligned with ScheduleResultMM.

T2M then parses the text output into ScheduleResultMM, with validation against allocation correctness and regulation compliance.

We will use Eclipse Modeling Tools with EMF/Ecore will be used for metamodels and models, QVTo for M2M transformations, and Acceleo for M2T transformations.

Creating and Updating the Models

Manual Creation:

AirportOps model: manually authored plus an import helper for a CSV flight plan.

Layout model: manually authored graph with assigned travel times.

Equipment model: manually authored fleet and shifts; or imported from CSV.

Automated Generation:

ScheduleProblemMM is M2M derived, and ScheduleResultMM is T2M parsed from solver output.

Transformations

List of Transformations

3 transformations, as illustrated in Figure 1, will be implemented. Their details are given below.

Domain-to-Problem M2M

Input: AirportOpsMM + EquipmentMM + LayoutMM.

Output: ScheduleProblemMM.

Responsibilities:

Expand each flight into concrete Jobs with time windows, durations, and sequence.

Map TaskTypes to ResourceTypes; list eligible GSEUnits per Job.

Compute travel-time function between Jobs by shortest paths on LayoutMM.

Encode regulatory exclusions (e.g., fuelling/catering exclusion).

Problem Serialization M2T

Input: ScheduleProblemMM.

Output: solver_input.json and a shell/batch script to invoke the solver.

Responsibilities:

Export jobs, windows, durations, sequences, resource pools, travel/setup times, and objectives in the solver's schema.

Tool Result Parsing T2M

Input: solver_result.json.

Output: ScheduleResultMM.

Responsibilities:

Parse assignments, start/end times, routes, etc.

Validate integrity (regulations, overlaps, sequence).

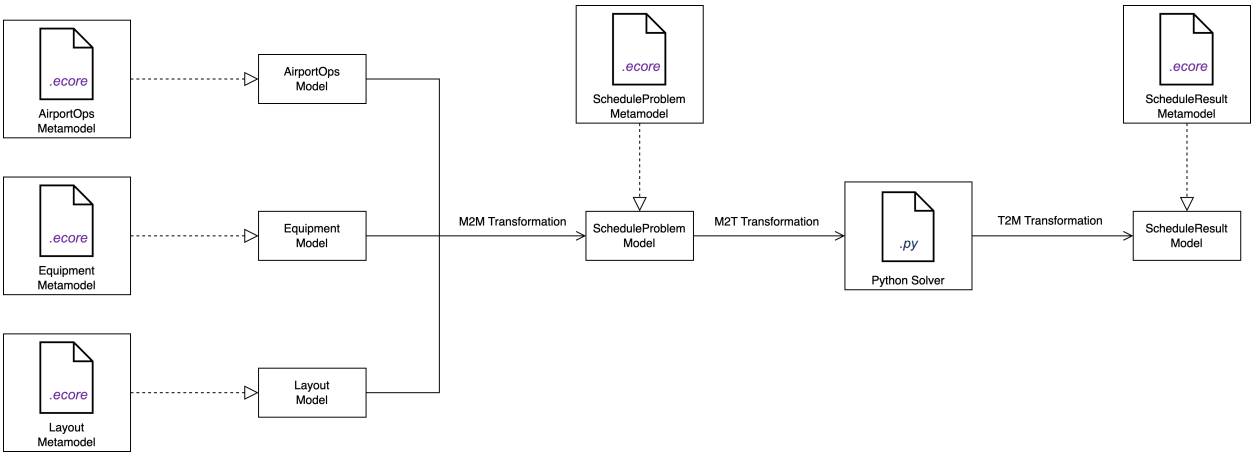

Figure 1. Overview of Transformations to be Implemented

Combining Transformations

Domain-to-Problem M2M (AirportOps + Equipment + Layout → ScheduleProblem).

Problem Serialization M2T (ScheduleProblem → solver_input).

External solver execution (consumes solver_input, produces solver_result).

Tool Result Parsing T2M (solver_result → ScheduleResult).

The transformations are combined in a simple sequence that runs on demand or when events occur. First, the Domain-to-Problem M2M combines AirportOpsMM, EquipmentMM, and LayoutMM into a ScheduleProblemMM for the chosen time frame. Next, the Problem Serialization M2T writes solver_input.json and runs the solver. The external solver returns solver_result.json. The Tool Result Parsing T2M reads that file, builds a ScheduleResultMM, and checks IDs, precedence, overlaps, and travel feasibility.